

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Алгоритмы и структуры данных»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

«Сортировка распределением (карманная сортировка) в очереди на
базе связного списка. Поиск подстрок методом Кнута-Морриса-Пратта»

Выполнил:

Пахомов Владимир Юрьевич, студент группы N3250

(подпись)

Проверил:

Ерофеев Сергей Анатольевич

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	2
ВВЕДЕНИЕ.....	3
1. АЛГОРИТМ СОРТИРОВКИ И ПОИСКА.....	4
2. ОПИСАНИЕ ИСПОЛЬЗУЕМЫХ ПЕРЕМЕННЫХ.....	5
3. БЛОК-СХЕМА АЛГОРИТМА.....	8
4. ТЕСТИРОВАНИЕ АЛГОРИТМА.....	8
5. ИСХОДНЫЙ КОД АЛГОРИТМА.....	9
ЗАКЛЮЧЕНИЕ.....	15
ПРИЛОЖЕНИЕ А.....	16

ВВЕДЕНИЕ

Цель работы — разработать программу карманной сортировки последовательности вещественных чисел, хранящейся в очереди на базе связного списка, записать результат её работы в красно-чёрное дерево, а также предусмотреть поиск подстроки методом Кнута-Морриса-Пратта

1. АЛГОРИТМ СОРТИРОВКИ И ПОИСКА

Программа получает на вход название файла, в котором хранятся исходные данные. Содержимое файла считывается целиком в динамический текстовый буфер (строку). Из этой строки последовательно считываются и преобразуются вещественные числа, которые затем записываются в односвязную очередь.

Далее происходит сортировка очереди по следующему алгоритму:

1. Создаются карманы (тоже отдельные очереди), количество карманов равно количеству элементов в очереди;
2. Далее элементы распределяются по карманам. Для начала определяются максимальный (max) и минимальный (min) элементы очереди. Индекс нужного кармана для каждого элемента определяется по формуле:
$$ind = \frac{(val - min) \cdot (numBuckets - 1)}{max - min}$$
3. Каждый заполненный карман сортируется на месте с помощью сортировки вставками;
4. Содержимое карманов последовательно сливается в одну очередь путём перемещения элементов.

Для поиска подстроки файл воспринимается как единая строка. Поиск подстроки выполняется методом Кнута-Морриса-Пратта. Алгоритм состоит из следующих этапов:

5. Построение префикс-функции (таблицы префиксов) для искомой подстроки. Для каждой позиции вычисляется длина наибольшего собственного префикса, совпадающего с суффиксом рассматриваемой части строки.
6. Последовательный просмотр текста. При несовпадении символов алгоритм использует значения префикс-функции для определения следующей позиции сравнения без повторной проверки уже обработанных символов.

Благодаря использованию префикс-функции количество сравнений существенно уменьшается по сравнению с наивным методом поиска.

2. ОПИСАНИЕ ИСПОЛЬЗУЕМЫХ ПЕРЕМЕННЫХ

Ниже представлена таблица с описанием использованных переменных.

Название	Тип	Границы типа	Назначение
value	float	от -3,4E+38 до 3,4E+38	Значение узла очереди
next	struct QNode*	-	Указатель на следующий узел очереди
head	struct QNode*	-	Указатель на первый элемент очереди
tail	struct QNode*	-	Указатель на последний элемент очереди
size	size_t	от 0 до $2^{64}-1$	Количество элементов в очереди
q	struct Queue*	-	Указатель на структуру очереди
node	struct QNode*	-	Указатель на создаваемый узел очереди
old	struct QNode*	-	Указатель на удаляемый узел очереди
cur	struct QNode*	-	Указатель на текущий элемент очереди
min	float	от -3,4E+38 до 3,4E+38	Минимальное значение в очереди
max	float	от -3,4E+38 до 3,4E+38	Максимальное значение в очереди
nb	int	от 0 до 2147483647	Количество корзин при карманной сортировке
buckets	struct Queue*	адрес памяти	Массив корзин для карманной сортировки
idx	int	от 0 до 2147483647	Индекс корзины для элемента
val	float	от -3,4E+38 до 3,4E+38	Временное значение элемента очереди
m	size_t	от 0 до $2^{64}-1$	Размер корзины или длина шаблона
n	size_t	от 0 до $2^{64}-1$	Размер очереди или длина текста
k	size_t	от 0 до $2^{64}-1$	Счётчик при построении префикс-функции
j	int	от -2147483648 до 2147483647	Индекс при сортировке вставками
pi	int*	адрес памяти	Таблица префиксов алгоритма КМП
q	size_t	от 0 до $2^{64}-1$	Текущая длина совпавшего префикса
pattern	char[256]	до 255 символов	Искомая подстрока

Название	Тип	Границы типа	Назначение
text	const char*	адрес памяти	Текст для поиска подстроки
pos	int	от -1 до 2147483647	Позиция найденной подстроки
result	int	от -1 до 2147483647	Результат поиска подстроки
key	float	от -3,4E+38 до 3,4E+38	Ключ узла красно-чёрного дерева
color	enum Color	RED, BLACK	Цвет узла красно-чёрного дерева
left	struct RBNode*	-	Указатель на левого потомка
right	struct RBNode*	-	Указатель на правого потомка
parent	struct RBNode*	-	Указатель на родительский узел
root	struct RBNode*	-	Корень красно-чёрного дерева
nil	struct RBNode*	-	Сторожевой NIL-узел дерева
t	struct RBTree*	-	Указатель на структуру дерева
z	struct RBNode*	-	Вставляемый узел дерева
x	struct RBNode*	-	Рабочий указатель при операциях дерева
y	struct RBNode*	-	Вспомогательный указатель при операциях дерева
newPrefix	char[512]	до 511 символов	Формируемый префикс для вывода дерева
prefix	const char*	адрес памяти	Текущий префикс при выводе дерева
isLeft	int	0 или 1	Признак левого поддеревья при выводе
fileContent	char*	адрес памяти	Буфер с содержимым файла
filename	char*	адрес памяти	Имя входного файла
file	FILE*	-	Файловый поток для чтения файла
buf	char*	адрес памяти	Буфер для чтения содержимого файла
ptr	const char*	адрес памяти	Текущая позиция в буфере при разборе файла
tmp	float	от -3,4E+38 до 3,4E+38	Временная переменная для чтения чисел
offset	int	от 0 до 2147483647	Количество обработанных символов функцией sscanf
argc	int	от 1 до 2147483647	Количество аргументов командной строки
argv	char**	адрес памяти	Массив аргументов командной строки

Название	Тип	Границы типа	Назначение
tree	struct RBTree	-	Красно-чёрное дерево
tmp (очередь)	struct Queue	-	Временная очередь для переноса данных
i	int	от 0 до 2147483647	Счётчик циклов

3. БЛОК-СХЕМА АЛГОРИТМА

Блок-схема алгоритма представлена в Приложении А.

4. ТЕСТИРОВАНИЕ АЛГОРИТМА

```
ЛР4 > ./lab4 data.txt
Исходный текст файла:
3 43 4325 1 46 12.44 -123.34 21947839 300 1 -200 -100 200 1000

Lorem ipsum dolor sit amet consectetur adipisicing elit. Voluptas temporibus
dolores animi obcaecati omnis aspernatur ratione nam ipsa, mollitia, unde
reiciendis. Magni nesciunt eos dignissimos quidem dolore iure assumenda saepe.

--- Исходные данные очереди ---
[ 3 -> 43 -> 4325 -> 1 -> 46 -> 12.44 -> -123.34 -> 2.19478e+07 -> 300 -> 1 -> -200 -> -100 -> 200 -> 1000 ] (size=14)
Введите подстроку для поиска методом Кнута-Морриса-Пратта: eos
Поиск подстроки "eos" (КМП)...
Подстрока найдена на позиции: 242

--- Отсортированные данные очереди ---
[ -200 -> -123.34 -> -100 -> 1 -> 1 -> 3 -> 12.44 -> 43 -> 46 -> 200 -> 300 -> 1000 -> 4325 -> 2.19478e+07 ] (size=14)

--- Красно-чёрное дерево (in-order, формат: значение [цвет]) ---
      [R] 2.19478e+07
     [B] 4325
    [R] 1000
   [B] 300
  [B] 200
 [B] 46
[R] 43
 [B] 12.44
 [B] 3
 [B] 1
[B] 1
 [B] -100
 [B] -123.34
 [B] -200
```

Рисунок 1 — Результат успешной работы алгоритма

```
ЛР4 > ./lab4
Использование: ./lab4 <имя_файла> [подстрока]
ЛР4 > ./lab4 data1.txt
Ошибка: не удалось открыть файл data1.txt
```

Рисунок 2 — Результат работы алгоритма при неуказанном входном файле или ошибке при открытии файла

5. ИСХОДНЫЙ КОД АЛГОРИТМА

Программа написана на языке C, для компиляции использовался компилятор gcc.

Исходный код Makefile:

```
1 | CC = gcc
2 | CFLAGS = -O3 -Wall -Wextra -Werror -fanalyzer
3 | TARGET = lab4
4 |
5 | all: $(TARGET)
6 |
7 | $(TARGET): lab4.c
8 |     $(CC) $(CFLAGS) $(TARGET).c -o $(TARGET)
9 |
10 | clean:
11 |     rm -f $(TARGET)
```

Исходный код lab4.c:

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | #include <string.h>
4 | #include <stdbool.h>
5 |
6 |
7 | typedef struct QNode {
8 |     float value;
9 |     struct QNode *next;
10 | } QNode;
11 |
12 | typedef struct Queue {
13 |     QNode *head;
14 |     QNode *tail;
15 |     size_t size;
16 | } Queue;
17 |
18 | void qInit(Queue *q) {
19 |     q->head = q->tail = NULL;
20 |     q->size = 0;
21 | }
22 |
23 | void enqueue(Queue *q, float value) {
24 |     QNode *node = malloc(sizeof(QNode));
25 |     if (!node) {
26 |         fprintf(stderr, "Ошибка: не удалось выделить память для узла очереди.\n");
27 |         exit(1);
28 |     }
29 |     node->value = value;
30 |     node->next = NULL;
31 |     if (q->size == 0) {
32 |         q->head = q->tail = node;
33 |     } else {
34 |         q->tail->next = node;
35 |         q->tail = node;
36 |     }
37 |     q->size++;
38 | }
39 |
40 | bool dequeue(Queue *q, float *out) {
41 |     if (q->size == 0) return false;
42 |     QNode *old = q->head;
43 |     *out = old->value;
44 |     q->head = old->next;
45 |     if (q->head == NULL) q->tail = NULL;
46 |     free(old);
```

```

47     q->size--;
48     return true;
49 }
50
51 void printQ(const Queue *q) {
52     if (q->size == 0) { printf("[ пусто ]\n"); return; }
53     QNode *cur = q->head;
54     printf("[ ");
55     while (cur) {
56         printf("%g", cur->value);
57         if (cur->next) printf(" -> ");
58         cur = cur->next;
59     }
60     printf(" ] (size=%zu)\n", q->size);
61 }
62
63 void insertionSortQueue(Queue *q)
64 {
65     if (q->size <= 1)
66         return;
67
68     QNode *sorted = NULL;
69     QNode *cur = q->head;
70
71     while (cur != NULL) {
72         QNode *next = cur->next;
73
74         if (sorted == NULL || cur->value <= sorted->value) {
75             cur->next = sorted;
76             sorted = cur;
77         }
78         else {
79             QNode *tmp = sorted;
80
81             while (tmp->next != NULL &&
82                 tmp->next->value < cur->value)
83             {
84                 tmp = tmp->next;
85             }
86
87             cur->next = tmp->next;
88             tmp->next = cur;
89         }
90
91         cur = next;
92     }
93
94     q->head = sorted;
95
96     q->tail = sorted;
97     while (q->tail != NULL &&
98         q->tail->next != NULL)
99     {
100         q->tail = q->tail->next;
101     }
102 }
103
104 void bucketSort(Queue *q) {
105     size_t n = q->size;
106     if (n <= 1) return;
107
108     float min, max;
109     QNode *cur = q->head;
110
111     min = max = cur->value;
112     cur = cur->next;
113
114     while (cur) {
115         if (cur->value < min) min = cur->value;
116         if (cur->value > max) max = cur->value;
117         cur = cur->next;

```

```

118     }
119     if (min == max) return;
120
121     int nb = (int)n;
122     Queue *buckets = malloc(nb * sizeof(Queue));
123     if (!buckets) {
124         fprintf(stderr, "Ошибка: не удалось выделить память для корзин.\n");
125         exit(1);
126     }
127     for (int i = 0; i < nb; i++) qInit(&buckets[i]);
128
129     float val;
130     while (dequeue(q, &val)) {
131         int idx = (int)((val - min) * (nb - 1) / (max - min));
132         if (idx < 0) idx = 0;
133         if (idx >= nb) idx = nb - 1;
134         enqueue(&buckets[idx], val);
135     }
136
137     for (int i = 0; i < nb; i++) {
138         if (buckets[i].size == 0)
139             continue;
140
141         insertionSortQueue(&buckets[i]);
142
143         while (buckets[i].size > 0) {
144             dequeue(&buckets[i], &val);
145             enqueue(q, val);
146         }
147     }
148     free(buckets);
149 }
150
151 static int *buildPrefixTable(const char *pattern, size_t m) {
152     int *pi = malloc(m * sizeof(int));
153     if (!pi) { fprintf(stderr, "Ошибка malloc КМП.\n"); exit(1); }
154     pi[0] = 0;
155     size_t k = 0;
156     for (size_t i = 1; i < m; i++) {
157         while (k > 0 && pattern[k] != pattern[i]) k = (size_t)pi[k - 1];
158         if (pattern[k] == pattern[i]) k++;
159         pi[i] = (int)k;
160     }
161     return pi;
162 }
163
164 int kmpSearch(const char *text, const char *pattern) {
165     size_t n = strlen(text);
166     size_t m = strlen(pattern);
167     if (m == 0) return 0;
168     if (m > n) return -1;
169
170     int *pi = buildPrefixTable(pattern, m);
171     size_t q = 0;
172     int result = -1;
173     for (size_t i = 0; i < n; i++) {
174         while (q > 0 && pattern[q] != text[i]) q = (size_t)pi[q - 1];
175         if (pattern[q] == text[i]) q++;
176         if (q == m) {
177             result = (int)(i - m + 1);
178             break;
179         }
180     }
181     free(pi);
182     return result;
183 }
184
185
186 typedef enum { RED, BLACK } Color;
187
188 typedef struct RBNode {

```

```

189     float key;
190     Color color;
191     struct RBNode *left, *right, *parent;
192 } RBNode;
193
194 typedef struct RBTree {
195     RBNode *root;
196     RBNode *nil;
197 } RBTree;
198
199 static RBNode *rbNewNode(RBTree *t, float key) {
200     RBNode *n = malloc(sizeof(RBNode));
201     if (!n) { fprintf(stderr, "Ошибка malloc RBNode.\n"); exit(1); }
202     n->key = key;
203     n->color = RED;
204     n->left = n->right = n->parent = t->nil;
205     return n;
206 }
207
208 void rbInit(RBTree *t) {
209     t->nil = malloc(sizeof(RBNode));
210     if (!t->nil) { fprintf(stderr, "Ошибка malloc nil.\n"); exit(1); }
211     t->nil->color = BLACK;
212     t->nil->key = 0;
213     t->nil->left = t->nil->right = t->nil->parent = t->nil;
214     t->root = t->nil;
215 }
216
217 static void leftRotate(RBTree *t, RBNode *x) {
218     RBNode *y = x->right;
219     x->right = y->left;
220     if (y->left != t->nil) y->left->parent = x;
221     y->parent = x->parent;
222     if (x->parent == t->nil) t->root = y;
223     else if (x == x->parent->left) x->parent->left = y;
224     else x->parent->right = y;
225     y->left = x;
226     x->parent = y;
227 }
228
229 static void rightRotate(RBTree *t, RBNode *x) {
230     RBNode *y = x->left;
231     x->left = y->right;
232     if (y->right != t->nil) y->right->parent = x;
233     y->parent = x->parent;
234     if (x->parent == t->nil) t->root = y;
235     else if (x == x->parent->right) x->parent->right = y;
236     else x->parent->left = y;
237     y->right = x;
238     x->parent = y;
239 }
240
241 static void rbInsertFixup(RBTree *t, RBNode *z) {
242     while (z->parent->color == RED) {
243         if (z->parent == z->parent->parent->left) {
244             RBNode *y = z->parent->parent->right;
245             if (y->color == RED) {
246                 z->parent->color = BLACK;
247                 y->color = BLACK;
248                 z->parent->parent->color = RED;
249                 z = z->parent->parent;
250             } else {
251                 if (z == z->parent->right) {
252                     z = z->parent;
253                     leftRotate(t, z);
254                 }
255                 z->parent->color = BLACK;
256                 z->parent->parent->color = RED;
257                 rightRotate(t, z->parent->parent);
258             }
259         } else {

```

```

260         RBNode *y = z->parent->parent->left;
261         if (y->color == RED) {
262             z->parent->color = BLACK;
263             y->color = BLACK;
264             z->parent->parent->color = RED;
265             z = z->parent->parent;
266         } else {
267             if (z == z->parent->left) {
268                 z = z->parent;
269                 rightRotate(t, z);
270             }
271             z->parent->color = BLACK;
272             z->parent->parent->color = RED;
273             leftRotate(t, z->parent->parent);
274         }
275     }
276 }
277 t->root->color = BLACK;
278 }
279
280 void rbInsert(RBTree *t, float key) {
281     RBNode *z = rbNewNode(t, key);
282     RBNode *y = t->nil;
283     RBNode *x = t->root;
284     while (x != t->nil) {
285         y = x;
286         if (z->key < x->key) x = x->left;
287         else x = x->right;
288     }
289     z->parent = y;
290     if (y == t->nil) t->root = z;
291     else if (z->key < y->key) y->left = z;
292     else y->right = z;
293     rbInsertFixup(t, z);
294 }
295
296 static void rbPrintHelper(const RBTree *t,
297                          const RBNode *n,
298                          const char *prefix,
299                          int isLeft)
300 {
301     if (n == t->nil)
302         return;
303
304     char newPrefix[512];
305
306     snprintf(newPrefix, sizeof(newPrefix), "%s%s",
307             prefix, isLeft ? "| " : " ");
308
309     rbPrintHelper(t, n->right, newPrefix, 0);
310
311     printf("%s%s %g\n",
312         prefix,
313         isLeft ? "└─" : "┬─",
314         n->color == RED ? "[R]" : "[B]",
315         n->key);
316
317     snprintf(newPrefix, sizeof(newPrefix), "%s%s",
318             prefix, isLeft ? " " : "| ");
319
320     rbPrintHelper(t, n->left, newPrefix, 1);
321 }
322
323 void rbPrint(const RBTree *t) {
324     if (t->root == t->nil) { printf("(дерево пусто)\n"); return; }
325     rbPrintHelper(t, t->root->right, " ", 0);
326     printf("%s %g\n",
327         t->root->color == RED ? "[R]" : "[B]",
328         t->root->key);
329     rbPrintHelper(t, t->root->left, " ", 1);
330 }

```

```

331 static void rbFreeNode(RBTree *t, RBNode *n) {
332     if (n == t->nil) return;
333     rbFreeNode(t, n->left);
334     rbFreeNode(t, n->right);
335     free(n);
336 }
337
338 void rbFree(RBTree *t) {
339     rbFreeNode(t, t->root);
340     free(t->nil);
341     t->root = t->nil = NULL;
342 }
343
344 char *readFileToString(const char *filename) {
345     FILE *f = fopen(filename, "r");
346     if (!f) {
347         fprintf(stderr, "Ошибка: не удалось открыть файл %s\n", filename);
348         exit(1);
349     }
350     fseek(f, 0, SEEK_END);
351     long len = ftell(f);
352     fseek(f, 0, SEEK_SET);
353     char *buf = malloc(len + 1);
354     if (!buf) { fclose(f); fprintf(stderr, "Ошибка malloc.\n"); exit(1); }
355     size_t rb = fread(buf, 1, len, f);
356     buf[rb] = '\0';
357     fclose(f);
358     return buf;
359 }
360
361 void parseBufferToQueue(const char *buf, Queue *q) {
362     const char *ptr = buf;
363     float tmp;
364     int offset;
365     while (sscanf(ptr, "%f%n", &tmp, &offset) == 1) {
366         enqueue(q, tmp);
367         ptr += offset;
368     }
369 }
370
371 int main(int argc, char **argv) {
372     if (argc < 2) {
373         fprintf(stderr, "Использование: %s <имя_файла> [подстрока]\n", argv[0]);
374         return 1;
375     }
376     char *fileContent = readFileToString(argv[1]);
377
378     Queue q;
379     qInit(&q);
380     parseBufferToQueue(fileContent, &q);
381
382     printf("Исходный текст файла:\n%s\n", fileContent);
383     printf("--- Исходные данные очереди ---\n");
384     printQ(&q);
385
386     char pattern[256];
387     if (argc >= 3) {
388         strncpy(pattern, argv[2], sizeof(pattern) - 1);
389         pattern[sizeof(pattern) - 1] = '\0';
390     } else {
391         printf("Введите подстроку для поиска методом Кнута-Морриса-Пратта: ");
392         if (fgets(pattern, sizeof(pattern), stdin))
393             pattern[strcspn(pattern, "\n")] = '\0';
394     }
395
396     printf("Поиск подстроки \"%s\" (КМП)... \n", pattern);
397     int pos = kmpSearch(fileContent, pattern);
398     if (pos != -1) printf("Подстрока найдена на позиции: %d\n", pos);

```

```

402 |         else                printf("Подстрока не найдена.\n");
403 |
404 |         bucketSort(&q);
405 |         printf("\n--- Отсортированные данные очереди ---\n");
406 |         printQ(&q);
407 |
408 |         RBTree tree;
409 |         rbInit(&tree);
410 |
411 |         float val;
412 |         Queue tmp;
413 |         qInit(&tmp);
414 |         QNode *cur = q.head;
415 |         while (cur) { enqueue(&tmp, cur->value); cur = cur->next; }
416 |
417 |         while (dequeue(&tmp, &val)) rbInsert(&tree, val);
418 |
419 |         printf("\n--- Красно-чёрное дерево (in-order, формат: значение [цвет]) ---\n");
420 |         rbPrint(&tree);
421 |
422 |         free(fileContent);
423 |         while (dequeue(&q, &val)) {}
424 |         rbFree(&tree);
425 |
426 |         return 0;
427 |     }

```

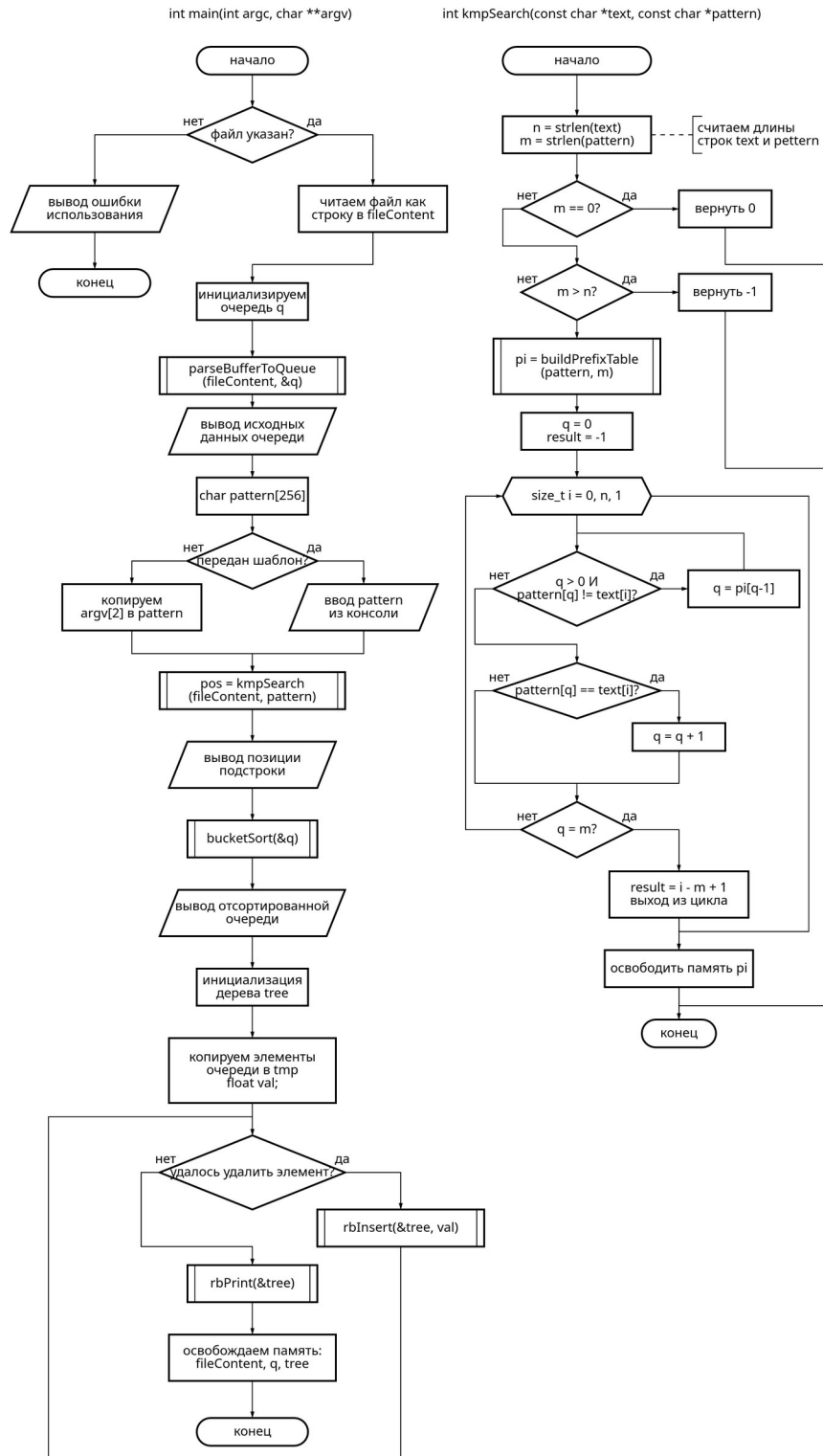
ЗАКЛЮЧЕНИЕ

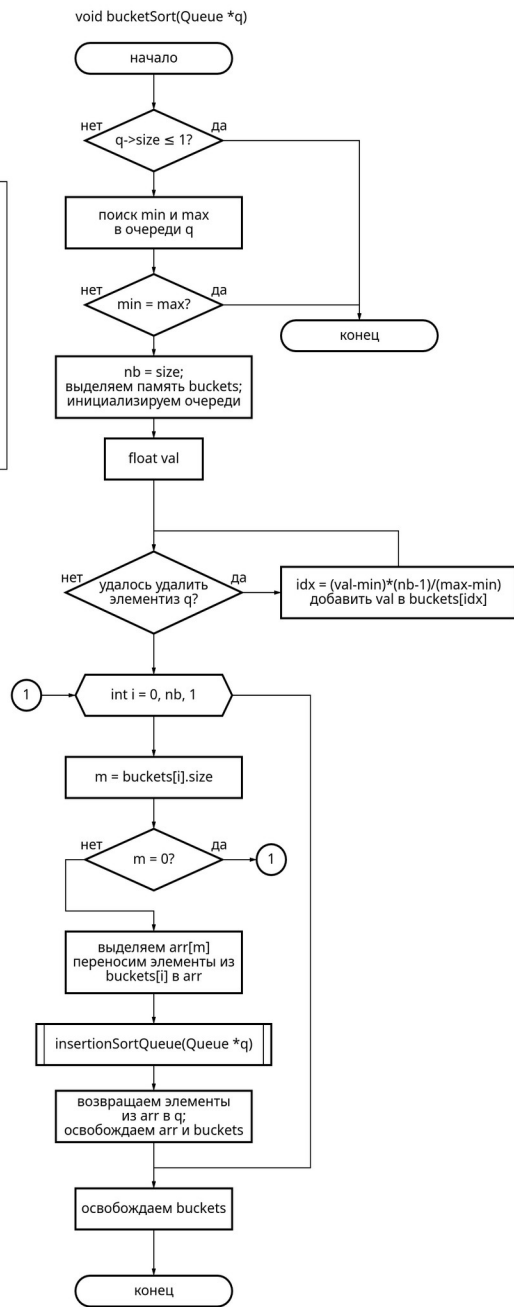
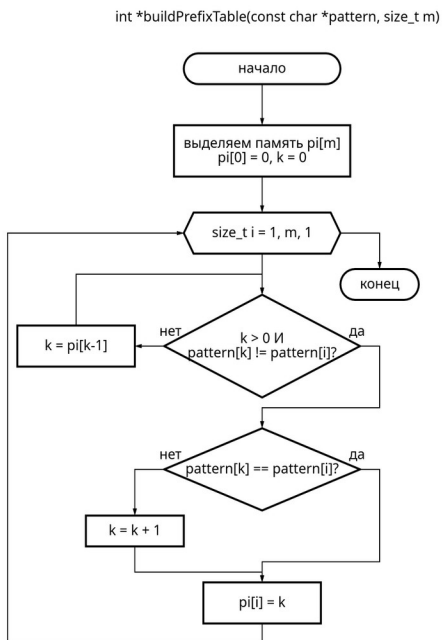
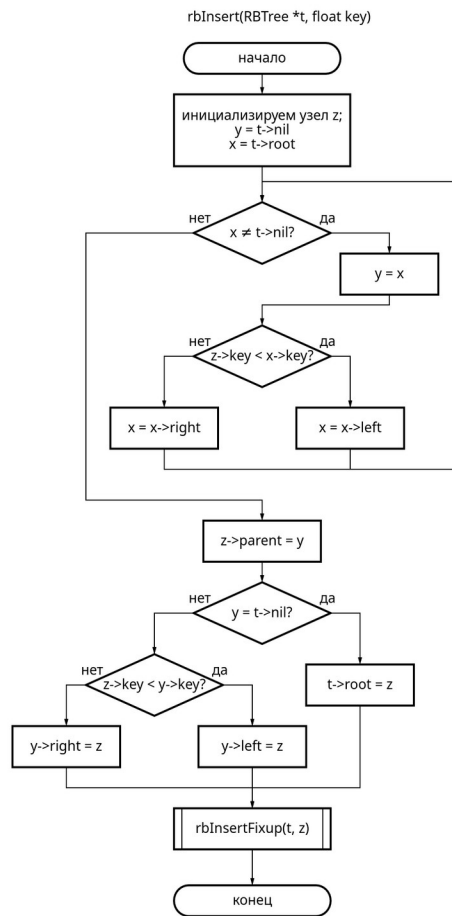
В ходе выполнения лабораторной работы была разработана программа на языке C для сортировки последовательности чисел с помощью алгоритма распределения (карманная сортировка), использующего очередь, построенную на базе связанного списка. Для решения задачи также был реализован алгоритм сортировки вставками. Числа считывались из файла и хранились в очереди, а результат записывался в красно-чёрное дерево. Помимо этого был реализован поиск подстроки в файле методом Кнута-Морриса-Пратта.

Программа разрабатывалась в среде VS Code, компилировалась с помощью gcc и запускалась в ОС Arch Linux.

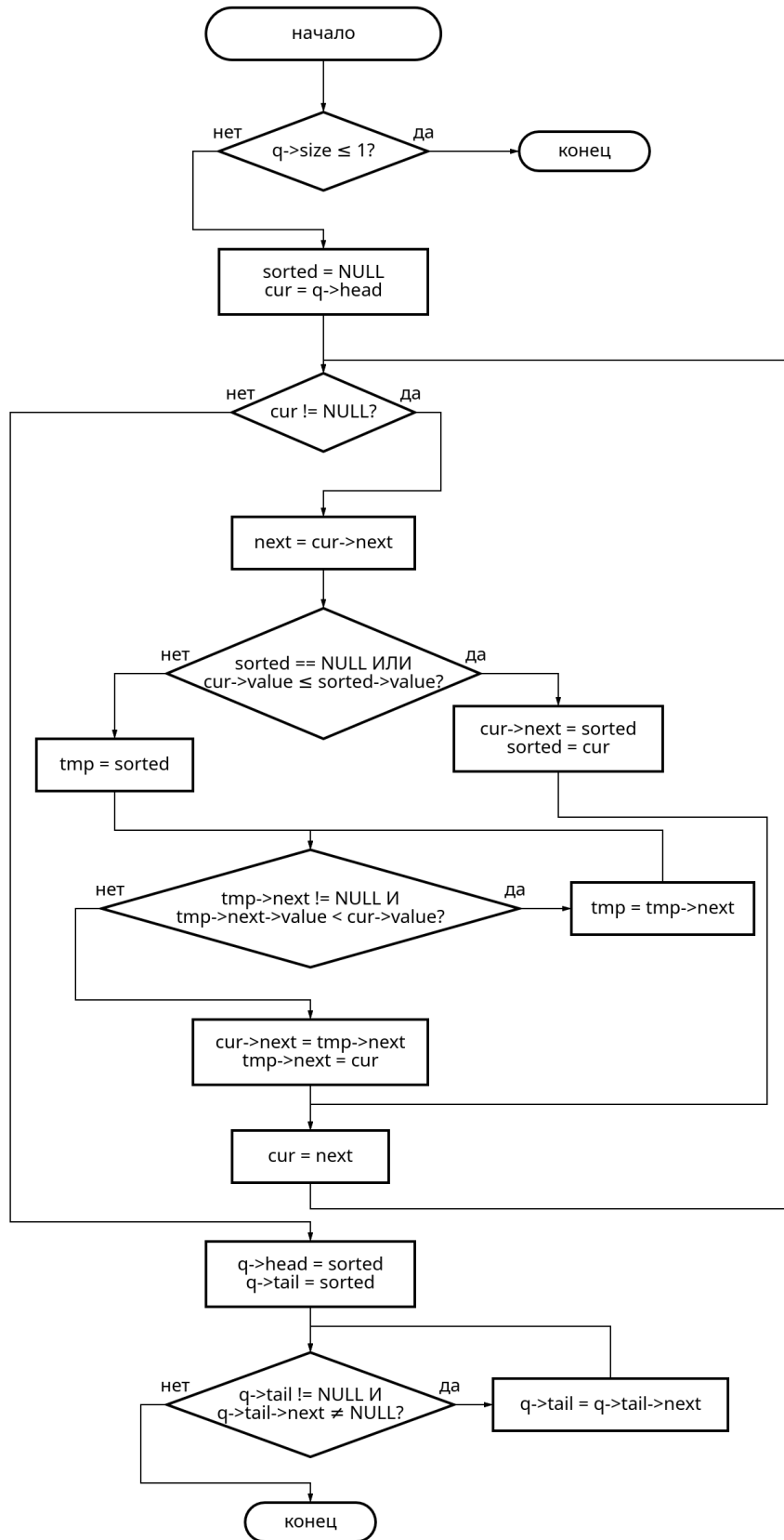
Тестирование алгоритма показало его корректную работу при различных входных данных. Выполнение работы позволило закрепить навыки разработки алгоритмов сортировки и поиска подстрок, а также углубить знания в области структур данных.

ПРИЛОЖЕНИЕ А

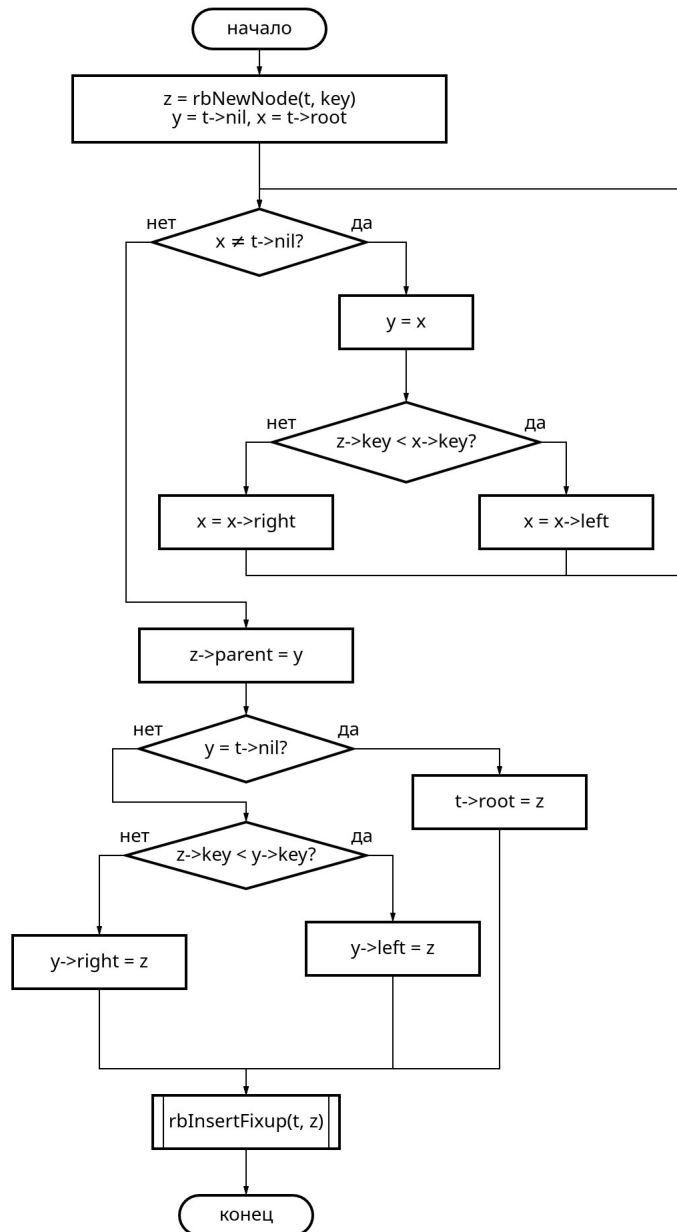




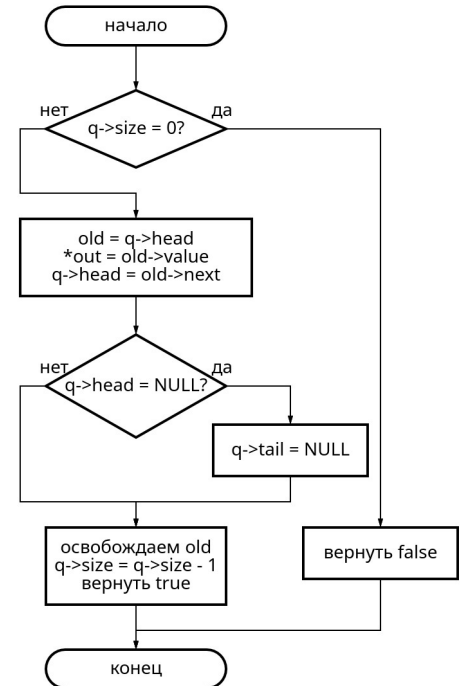
void insertionSortQueue(Queue *q)



Вставка в КЧ-дерево
void rbInsert(RBTree *t, float key)



bool dequeue(Queue *q, float *out)



void enqueue(Queue *q, float value)

